

Meridian / Replenishment Control Tower — Complete Deep Dive

What this document is: A presentation-ready explanation of the entire Meridian project — what it does, how it works, every technical concept explained in plain language. Written so you can walk someone through it end to end.

Part 1: The Business Problem (Why This Exists)

Flipkart Minutes runs roughly 1,000 **dark stores** (small warehouses that only do delivery — no walk-in customers. Think of them as mini fulfilment centres scattered across cities). These dark stores are stocked by an automated engine called **IWIT** (Inventory & Warehouse Inbound Transfer — the system that decides what products to send where).

The problem: when a product goes **OOS** (Out Of Stock — literally zero available units at a dark store), there's currently no single place to find out *why*. Was it because the planning system didn't send enough? Was it because trucks lost units in transit? Was it because demand spiked unexpectedly?

Today, people use scattered Excel sheets, manual CSVs, and siloed tools. The **Replenishment Control Tower** (codename: **Meridian**) is a single dashboard product that gives a unified view of:

- What % of products are in stock across all dark stores
 - Which products are out of stock and *why*
 - Trends over time (getting better or worse?)
-

Part 2: The Three Dashboards You're Building

Dashboard 1: Instock Monitoring (the main landing page)

This is the "control room" view. Imagine opening it and seeing:

Top tiles showing:

- Overall Instock % (e.g. "82.3% of all FSN×DS pairs have stock"). **FSN** = Flipkart Serial Number — a unique product ID, like a barcode. **FSN×DS** = one specific product at one specific dark store.
- Number of OOS FSN×DS pairs (e.g. "14,200 products are missing at stores")
- Number of dark stores below 90% instock
- Each with **delta arrows** (comparison numbers) showing: vs last snapshot, vs day start (6 AM), vs last week

Below the tiles:

- **Worst FSN List** — the products with the most OOS across dark stores. Each row shows the product name, how many dark stores it's missing at, its instock %, and the **Top Root Cause** (which of the 4 buckets — explained in Part 5 — is the biggest reason it's missing).
- **DS < 90% List** — dark stores performing badly. Click one → see which FSNs are missing there.
- **Subpivots** — slice all of the above by BU (Business Unit, like "Grocery" or "Electronics"), SC (Super Category), Vertical, Brand, Sourcing Hub (the warehouse sending stock), or Network Component (the physical shipping route).

Every number on this dashboard is pre-computed by Spark (a big data processing engine — explained in Part 4). The API just fetches and returns it. No calculation happens when you open the page.

Dashboard 2: Instock Diagnostics (product drilldown)

This is the "detective" view. You pick one specific FSN + dark store from a dropdown → see the full RCA (Root Cause Analysis — the detailed breakdown of *why* it's OOS):

- Which **sourcing hub** (the upstream warehouse assigned to supply this product to this dark store) was supposed to send it
- The **waterfall report** (a step-by-step calculation the planning engine ran: how much stock is available → how much is in transit → how much was allocated → what constraints blocked it)
- Whether the **lead time** (the expected number of hours for stock to travel from source hub to dark store) has been exceeded
- The 4 root-cause bucket breakdowns
- Individual **STN** (Stock Transfer Note — a document/record created when a batch of products is supposed to move from one warehouse to another) details

Dashboard 3: Replenishment Monitoring

Less detailed in the current Confluence docs. Focuses on the replenishment cycle health — how well the automated engine is performing at keeping stores stocked. You mentioned this is one of the three, so it's part of the project scope.

Part 3: The Old System (zero-ops-skynet / IPC RCA) — What Existed Before

Before Meridian, there was the **IPC RCA** tool (the repo you saw earlier: `zero-ops-skynet`). Here's what it did and why it's being replaced:

How the old system worked:

1. An upstream pipeline (Spark jobs + Krios — Flipkart's internal data platform) produced one massive MySQL table: `(ipc_rca_sh_detail)` (millions of rows, one per FSN × dark store × sourcing hub)
2. A Spring Boot API read from that table — but here's the key problem: **every metric was computed at query time**. When you opened the UI, the API would query MySQL, join tables, calculate instock %, compute bucket percentages, etc. *on every request*.
3. Bulk export happened via Azkaban (a job scheduler — like cron but for big data pipelines) → Spark wrote CSVs to GCS (Google Cloud Storage — like a cloud hard drive) → users got a download link.

Why it's being replaced:

- Query-time computation = slow and fragile. If the table has 10M rows, every dashboard load is expensive.
- Only 3 root-cause buckets (SC/Demand/Cron were lumped together in ways that weren't clean).
- No delta tracking — you couldn't see "instock was 80% this morning, now it's 78%".
- No city-level or subpivot views — you could only look at FSN × dark store level.
- No Elasticsearch — so no fast search, no faceted filtering (no "show me only Grocery products that are OOS because of Supply Chain issues").

What's being reused from the old system:

- The RCA computation logic (the actual formulas for the 4 checks) — ported from `(iwit-spark)` into the new pipeline
 - The bulk export pattern (`(ipc_requests)` table + Azkaban + D42 presigned URLs)
 - The general idea of a browser UI talking to a Spring Boot API
-

Part 4: The New Architecture — 6 Layers Explained

The new system has 6 layers. Think of it as a conveyor belt: raw data enters at one end, gets cleaned, computed, stored, and finally served to your browser.

Layer 1: Data Sources (where raw data comes from)

There are 16 sources total, coming through 3 different paths:

Path A — Manual CSV uploads (5 sources) These are business files that humans upload periodically. They don't change often:

- `(sell_master)` — **THE UNIVERSE**. This file defines which products are sellable at which dark stores. If a product isn't in this file for a dark store, it's not even counted. This is the denominator for instock %.

- `replenishment_master` — maps each FSN to its sourcing hub(s), priority, and design lead time
- `city_mapping` — maps each dark store to a city (needed for the City rollup view)
- `network_component` — maps shipping routes to network components (for the NC subpivot)
- `demand_annotation` — flags for promos, new launches, seasonality, long-tail products (for Demand sub-buckets)

Path B — Production system facts (9 sources) These are live data from Flipkart's production databases:

- `IWIT_STN_Transact_fact` — STN header (when was the transfer created, source/destination, status)
- `IWIT_STN_Items_Transact_fact` — STN line items (the actual quantity numbers: reserved, packed, dispatched, accepted, and every drop-off stage). **This is the single most important fact table** — it powers both the SC and STN root-cause buckets.
- `inventory_snapshot` — current ATP (Available To Promise — how many units are physically at the store and ready to sell) per FSN per FC (Fulfilment Centre / warehouse)
- `waterfall_report` — the output of the planning engine's allocation run (explained in Part 5)
- `cp_bi_prod_sales__forward_unit_live_hourly_fact` — hourly sales data (actual units sold per FSN × dark store per hour)
- `consignments_fact` / `consignment_items_fact` — dispatch and acceptance records (did the truck actually arrive? were items accepted?)
- `fsn_taxonomy_details_fact` — product categorisation (BU/SC/Vertical/Brand)
- `retail_marketplace_alpha_product_data_fact` — product name (for display)
- `ekart_mdm_cluster_mapping_fact` — MDM (Master Data Management) table that maps clusters to FCs (needed because the waterfall report uses "cluster" IDs but STN facts use FC IDs — this table bridges them)

Path C — Pipeline CSVs (2 sources) These are outputs of existing upstream jobs:

- `inventory_snapshot` (Hive)
- `waterfall_report` (Hive, with a lookback window of multiple cron runs)

Layer 2: Ingress (how raw data gets into the lake)

What "ingress" means: the process of getting raw data from its source into a standardised format in the data lake.

Two mechanisms:

- **Zero-ETL** (file-trigger) — for CSVs. When a file lands on GCS, it triggers a load job that reads the CSV and writes it as an Iceberg table. "Zero-ETL" means no separate

Extract-Transform-Load pipeline — the data platform handles it directly.

- **Flink CDC** — for production facts. **Flink** is a stream processing engine (like Spark but for real-time data). **CDC** = Change Data Capture — it reads the database's change log (binlog/Kafka events) to capture inserts, updates, and deletes as they happen. Flink deduplicates them (keeps only the latest version of each row) and writes them to Iceberg. Freshness: ~5 minutes.

Layer 3: Iceberg Lake (the central data store)

What is Apache Iceberg?

Iceberg is NOT a database. It's NOT a compute engine. It's a **table format** — think of it as a smart filing system that sits on top of regular Parquet files (a columnar file format optimised for analytics — stores data in columns instead of rows, which makes queries that scan specific columns very fast) on GCS.

What makes Iceberg special is **snapshots**. Every time you write data to an Iceberg table, it creates an immutable (can't be changed) snapshot with a unique ID and a timestamp. That snapshot points to the exact set of data files that were valid at that moment.

Why snapshots matter (the 3 superpowers):

1. **Incremental reads** — Instead of reading all 430 million rows every 2 hours, the pipeline says "give me only the rows that changed since snapshot #1001." Iceberg knows exactly which files are new. Result: reads ~950K rows instead of 430M. This is done with `(start-snapshot-id)`.
2. **Time-travel** — You can read the table *as it was at midnight*. So to compute "vs day start," you don't need a separate copy of the data. You just say `(as-of-timestamp = today at 6 AM)` and Iceberg gives you back the data as it existed then. No self-joins (a self-join is when a table is joined with itself — usually slow and confusing), no duplicate tables.
3. **ACID writes** — ACID stands for Atomicity, Consistency, Isolation, Durability — basically guarantees that writes either fully succeed or fully fail, never leaving data in a half-written state. If the pipeline crashes halfway through writing, readers never see partial/corrupted data. This is done via `(overwrite-partitions)` (replaces an entire partition atomically — a partition is a subdivision of the table by a key like date or region).

What's in the lake:

- All the 16 source tables (converted to Iceberg)
- All the output tables produced by the compute pipeline (also Iceberg)
- So the outputs of one run become the inputs for the next run's time-travel comparison — a self-feeding loop

Layer 4: Compute Pipeline (the brain — Spark jobs)

What is Spark? Apache Spark is a distributed data processing engine. "Distributed" means it splits work across many machines in parallel. It's used for big data — when you have millions or billions of rows that one machine can't handle alone.

What is Azkaban? A job scheduler. Think of it as cron (the Linux scheduler that runs commands at fixed times) but designed for data pipelines. You define a DAG (Directed Acyclic Graph — a chain of jobs where each job depends on the previous one finishing) and Azkaban runs them in order.

The pipeline runs every ~2 hours, triggered by a watermark gate (doesn't start until source data is fresh enough). It has **9 nodes** (originally designed as 4, but split for clarity):

Node: `instock_prep` (**readiness gate**) A Python script that checks all 16 sources before anything runs. If any source is stale, has a config field still set to `TODD_*`, or is unreachable → the whole pipeline fails and pages on-call. This is important: the pipeline NEVER runs on partial data.

Node: `J1 instock_ingestion` (**the only node that touches raw sources**) Reads all 16 sources, scopes everything to the **Minutes universe** (only FSN×DS pairs that exist in `sell_master` — everything else is ignored), normalises column names and types, and writes **5 staged tables** (intermediate output tables that downstream nodes read from):

- `instock_stg_masters` — universe + dimension tables
- `instock_stg_sales` — hourly sales
- `instock_stg_inventory` — current ATP at stores
- `instock_stg_waterfall` — planning engine output
- `instock_stg_movement_inputs` — STN + consignment data

Why 5 separate tables? Because Azkaban nodes are separate OS processes — they can't share in-memory data. So J1 writes to disk, and J2/J3/J4 read what they need.

Node: `J2 instock_movement_fact` (**QUANTITIES — how much is stuck**) This is where the STN quantity waterfall is computed. For every FSN moving from a sourcing hub to a dark store, it traces the lifecycle:

```
reserved_qty (how much was supposed to move)
  = cons_item_accepted_quantity (actually arrived)
  + pending_to_pack (sitting in warehouse, not yet packed)
  + prepack_lost (lost before packing)
  + pre_dispatch_lost (lost after packing but before dispatch)
  + dispatch_pending (packed but truck hasn't left)
  + intransit_truck (on a truck right now)
  + intransit_lost (lost during transit)
  + rejected_at_ds (arrived but QC rejected)
  + destination_lost (lost at destination after arrival)
```

This is the **conservation identity** — all the numbers MUST add up to `reserved_qty`. If they don't, something is wrong with the data. J2 checks this and fails if there's a gap > 0.

Output: `iwit_movement_fact` (one row per FSN×DS×SH with all the quantity columns).

Node: `J3 instock_rca_computation` (**REASONS — why is it stuck**)

This is the brain. It reads the movement fact + all staged tables and runs the 4 root-cause checks (explained fully in Part 5).

Key distinction between J2 and J3 (the most confusing part of the pipeline):

- **J2 asks: HOW MUCH is stuck?** It reads numerical quantity columns (`reserved_qty`, `cons_item_accepted_quantity`, etc.)
- **J3 asks: WHY is it stuck?** It reads status string columns (`stn_status`, `consignment_status`, `failure_reason`) to classify each stuck unit into a reason bucket.

They both read from `stg_movement_inputs` but use completely different columns from it.

J3 produces **6 output tables**:

1. `instock_fsn_ds_sh_fact` — the main 31-column fact table (one row per FSN × dark store × sourcing hub, with all RCA fields)
2. `instock_fsn_ds_fact` — same but collapsed over sourcing hub (one row per FSN × DS)
3. `instock_cron_breakdown` — why cron under-allocated (19 constraint labels)
4. `instock_sc_breakdown` — which lifecycle stage each STN is stuck at (14 labels)
5. `instock_stn_creation_detail` — which of 35 failure stages caused TR→STN drop-off
6. `instock_inaccurate_norm_detail` — demand forecast accuracy per FSN×DS×SH

Node: `instock_rca_verification` (**data quality gate**) Runs 20 domain rules against J3's outputs. If any BLOCKER rule fails, J4/J5/J6 never run. This means bad data never reaches MySQL or Elasticsearch. Examples of rules: "SC% + STN% + Demand% + Cron% must

approximately sum to 100%", "no FSN×DS should appear twice", "instock % must be between 0 and 100".

Node: J4 instock_aggregation (**rollups**) Takes the FSN×DS×SH-level data and rolls it up to higher levels:

- FSN×DS → Dark Store (sum OOS FSNs, compute DS-level instock %)
- Dark Store → City (via city_mapping)
- Everything × subpivots (BU, SC, Vertical, Brand, Sourcing Hub, Network Component — each combination gets its own row)

Also computes:

- **OOS Breadth %** = what % of dark stores is this product OOS at
- **Intransit-Instock %** = what % of FSN×DS pairs have either stock OR an open STN in transit (a "softer" instock that counts stuff on the way)

Node: J5 instock_snapshot (**history + deltas**) Appends this run's aggregates as an immutable timestamped partition (tagged with refresh_id). Then computes deltas:

- **vs Last Snapshot** = current value – previous run's value
- **vs Day Start** = current value – first run's value at or after 6 AM IST today
- **vs Last Week** = current value – the ~6 AM run from 7 days ago

These deltas are stored as inline fields (extra columns on the same row) so the API doesn't need to compute them.

Node: anomaly_gate (**safety check**) Compares this run's overall instock % vs the previous snapshot. If the shift is larger than a configured threshold (e.g. instock drops 10% in one run, which probably means a data issue, not a real-world event), it writes a HOLD file and exits with code 43, which stops J6 from publishing. This prevents silently pushing a bad pipeline run to the dashboard.

Node: J6 instock_serving_sink (**dual-write to MySQL + Elasticsearch**) Takes the source-of-truth Iceberg/Krios tables and writes them to both serving stores:

- **MySQL:** full FSN×DS×SH detail including heavy JSON blobs (explained below)
- **Elasticsearch:** pre-aggregated, flattened rows for fast dashboard queries

The write is idempotent (can be re-run safely — it overwrites by refresh_id), so running it twice produces the same result). Both stores are **projections** (read-only copies shaped for their access pattern) of the Iceberg tables — either can be rebuilt from scratch by re-running J6.

Layer 5: Serving (MySQL + Elasticsearch)

Why two stores? They're not copies of each other. They store different data at different

granularity, optimised for different access patterns.

Elasticsearch (ES) A search engine optimised for fast filtered queries and aggregations.

What's in it:

- `instock_city_summary` — one row per city per run, with all the pre-computed metrics (instock %, DS<90% count, deltas, top root cause, bucket %)
- `instock_ds_summary` — one row per dark store per run
- `instock_fsn_ds_flat` — one row per FSN×DS, **denormalised** (denormalised = instead of having a separate taxonomy table and joining at query time, the BU/SC/Vertical/Brand/SH/NC values are copied directly onto every row so a single query can filter by any pivot without joins)
- `instock_overall_tile` — the headline numbers for the top of the page

Why ES:

- **Faceted filtering** = "show me all OOS FSNs in Bangalore where BU=Grocery and Root Cause=Supply Chain." ES does this natively with bool filters + terms aggregations (a terms aggregation groups rows by a field value and counts them — like SQL GROUP BY but much faster in ES).
- **Full-text search** = typing "Amul butter" in the FSN search box. ES tokenises the text and finds matches.
- **Aggregation buckets** = compute average instock % across 200 dark stores without scanning each row. ES builds in-memory bucket trees.

What's NOT in ES: the heavy JSON detail (constraint breakdowns, per-SH waterfall match details). Those are too large and wouldn't benefit from ES's indexing.

MySQL A relational database optimised for exact key lookups and structured data.

What's in it:

- `instock_fsn_ds_sh_fact` — the full-fidelity RCA detail. Includes fields like `constraint_array_cluster_raw` (a JSON blob — "blob" just means a large chunk of data stored as a single field, here it's a JSON string that can be hundreds of characters — containing the full list of constraints that limited allocation), `supply_chain_sh_breakdown` (JSON listing per-sourcing-hub losses), and all the individual drop-off quantity columns.
- `sc_breakdown` — reason × FSN×DS (one row per SC sub-reason: "Pending To Pack: 42 FSN×DS pairs, 3,100 impacted units")
- `stn_detail` — individual STN records with status, quantities, dates
- `demand_mape` — per FSN×DS demand accuracy numbers
- `ipc_requests` (reused from old system) — tracks async bulk CSV export jobs

Why MySQL:

- **Point lookups by primary key** — when you click one specific FSN × DS in the Diagnostics view, the API does `(SELECT * FROM instock_fsn_ds_sh_fact WHERE fsn=? AND dark_store=? AND sourcing_hub=?)`. MySQL answers this in milliseconds.
- **Heavy JSON storage** — MySQL doesn't care about the size of a JSON column; it stores it and returns it. ES would need to index it, which would be wasteful.
- **Source of truth for exports** — bulk CSV downloads read from MySQL.

The routing rule (how the API decides which store to query):

- Dashboard tiles, lists, faceted filters, search box → **Elasticsearch**
- Single-key RCA drilldown, diagnostics, STN detail rows → **MySQL**
- Bulk export: small (< threshold rows) → inline from MySQL; large → Spark job reading Iceberg → GCS CSV

Layer 6: REST API (`instockctl-api`)

A new **stateless** (no session data stored on the server — every request contains everything the server needs to respond. This means any replica can handle any request) Spring Boot microservice deployed on **Kubernetes** (K8s — a container orchestration platform that runs your app in Docker containers, auto-scales them, and restarts them if they crash).

The key design principle: the API does ZERO computation. Every number was already computed by Spark (Layer 4) and stored in MySQL/ES (Layer 5). The API is just a thin controller that translates an HTTP request into an ES/MySQL query and returns JSON. This is what makes it fast and scalable.

Deployment: 3 replicas minimum, auto-scales to 10 via HPA (Horizontal Pod Autoscaler — K8s watches CPU usage and adds more pods when it gets high).

Async CSV Export (the "Download" button)

Uses a **two-lane router** — the only genuinely new piece vs the old system:

1. User clicks Export → API inserts a row in `(ipc_requests)` with status=PENDING, returns a `(requestId)` immediately (the UI never blocks)
2. API does a cheap `(COUNT(*))` to estimate row count
3. **If small** (e.g. an RCA modal with 14 rows): an in-process worker thread in the API pod runs the query, writes CSV, uploads to GCS, updates the row to COMPLETED with a presigned download URL (a time-limited HTTPS link that lets the browser download directly from GCS without needing credentials)
4. **If large** (e.g. "Export All FSN×DS"): the API triggers an Azkaban flow → Spark reads Iceberg → writes CSV to GCS → updates the DB row → UI polls and gets the download URL

Why two lanes: running Spark for 14 rows wastes minutes of scheduling overhead. Running the API thread for a million rows would crash the pod with OOM (Out Of Memory — the process tries to use more RAM than the container has and gets killed).

Part 5: The 4 Root-Cause Buckets (the core logic)

These only apply to **OOS rows** (FSN×DS pairs where ATP = 0 at the dark store). A single FSN×DS can trigger multiple buckets simultaneously — they're independent checks. Each bucket gets a share of **impacted quantity** (the number of units affected), and the **Top Root Cause** = whichever bucket has the highest qty.

Bucket 1: Cron Under-Allocation (the planning engine didn't ask for enough)

What happens in the real world: Every few hours, the IWIT planning engine runs a "cron" (scheduled batch job). It looks at: how much stock does this dark store need (the `InventoryTarget`)? How much is already there (`filtered_atp`)? How much is on the way (`intransit`)? How much did it just allocate (`allocated_quantity`)? If $\text{filtered_atp} + \text{intransit} + \text{allocated_quantity} < \text{InventoryTarget}$, there's a deficit — the planning engine didn't fully cover the need.

Why it might happen: The allocation algorithm has constraints (business rules that limit what it can do):

- `EXCESS` — the source hub doesn't have enough stock
- `INVENTORY_AT_SRC` — the source hub has stock but below a threshold
- `CAPACITY_*` — the shipping route is at capacity
- `ACTIVE_SELLER` — no active seller for this product
- `CASE_SIZE` — minimum shipment quantity not met
- `NO_CONNECTED_SOURCE` — no sourcing hub mapped to this dark store for this FSN
- And ~13 more

The pipeline's `attributeConstraints()` function distributes the deficit across these constraints proportionally, using the `Constraint Array Cluster` field from the waterfall report.

The numbers:

- $\text{cron_deficit} = \text{InventoryTarget} - \text{filtered_atp} - \text{intransit}$
- Split across constraint reasons from the `Constraint Array Cluster` hierarchy

Bucket 2: STN Drop-Off (units lost at handoff, BEFORE physical movement)

What happens in the real world: The planning engine said "send 50 units." An STN was created. But something went wrong *before the truck left*:

- pending_to_pack — the STN was created but nobody packed the items yet
- prepack_lost — items were lost/damaged before packing
- pre_dispatch_lost — items were packed but lost before dispatch
- dispatch_pending — items are packed and waiting for a truck that hasn't left yet
- no_stn_created — the waterfall said to allocate but no STN was ever created ($= \max(0, \text{waterfall.allocated_quantity} - \sum \text{reserved_qty})$)

The numbers:

- $$\text{STN_impacted_qty} = \sum \text{pending_to_pack} + \sum \text{prepack_lost} + \sum \text{pre_dispatch_lost} + \sum \text{dispatch_pending} + \text{no_stn_created}$$

Bucket 3: Supply Chain (units lost DURING physical movement, after dispatch)

What happens in the real world: The truck left the source hub. But between dispatch and arrival, units were lost:

- intransit_truck — still on the truck (legitimate if within lead time, but counted as loss if lead time exceeded)
- intransit_lost — fell off the truck / went missing during transit
- rejected_at_ds — arrived at dark store but QC (Quality Check) rejected them (damaged, wrong items)
- destination_lost — arrived and accepted but then lost at the dark store (putaway error, theft)

The trigger condition: $\text{atp} + \text{theoretical_intransit} + \text{reco} > \text{anticipated_sales}$ (the system planned enough, so the shortfall isn't a planning issue) AND $\text{atp} + \text{actual_intransit} < \text{anticipated_sales}$ (but actual arrival fell short — something went wrong in transit).

The numbers:

- $$\text{SC_impacted_qty} = \sum \text{intransit_truck} + \sum \text{intransit_lost} + \sum \text{rejected_at_ds} + \sum \text{destination_lost}$$
- Roughly = $\sum \text{dispatched} - \sum \text{accepted}$

Bucket 4: Demand MAPE (demand exceeded forecast)

MAPE = Mean Absolute Percentage Error — a measure of how wrong the forecast was. If DRR (Daily Run Rate — expected units sold per day) predicted 10 sales but actual was 25, the MAPE is 150%.

What happens in the real world: The store had stock, the planning engine sent the right amount based on its forecast, transit worked fine... but demand spiked. Customers bought more than expected.

The numbers:

- $\text{Demand_impacted_qty} = \text{actual_sales} - \text{anticipated_sales}$
- Sub-buckets come from `demand_annotation`: was it a promo? a new launch? a seasonal spike? a long-tail product with noisy demand?

The SC vs STN Split (a key design decision)

The **old IPC RCA system** had only 3 buckets — all STN-related losses were lumped into "Supply Chain Issue." Meridian splits this into two non-overlapping siblings:

- **STN bucket** = pre-dispatch losses (units never left the source hub)
- **SC bucket** = post-dispatch losses (units left but didn't arrive intact)

This makes the quantity attribution clean — the conservation identity guarantees:

$\text{reserved_qty} = \text{accepted} + (\text{all STN-stage losses}) + (\text{all SC-stage losses})$. No double-counting.

Part 6: Architecture Decisions (from the branch comparison docx)

The docx you uploaded is a branch analysis comparing `master` and `main` in the meridian repo. Key findings:

The pipeline grew from 4 nodes to 9. The original mental model was: Node1 (Ingest) → Node2 (OOS+RCA) → Node3 (MySQL export) → Node4 (async CSV). It's now 9 nodes because:

- Node2 was split into J2 (quantities) and J3 (reasons) — they answer different questions
- Node3 became J6 (dual-writes to MySQL AND Elasticsearch, not just MySQL)
- Node4 (async CSV export) was removed from the DAG — it now happens on-demand from the API layer
- Verification gate, aggregation, snapshot, and anomaly gate were added as separate nodes

Key architecture decisions made during design:

Decision	What was decided	Why
ES populated by J6 Spark directly	NOT via elasticsearch-hadoop connector from Hive	ES and MySQL are populated in the same Spark run, so they never diverge
ES vs MySQL routing	ES = tiles/lists/facets (pre-aggregated). MySQL = full-fidelity RCA drilldowns + exports	They're not copies — different data at different grain
Manual CSVs from FDP direct	No Iceberg R1 layer for manual inputs	Simpler setup (but means no time-travel on these sources)
Spark only for ingestion	Structured Streaming for the sales fact, batch for everything else	Flink CDC was considered and rejected — Spark-only is simpler ops
RCA reuse	Port <code>IpccRcaComputationDriver</code> from <code>iwit-spark</code> , not rebuild	The existing logic in the <code>master</code> branch is proven and tested
4th RCA bucket (STN Drop-Off)	New in Meridian — old pipeline only had 3 buckets (SC/Demand/Cron)	STN = pre-dispatch losses; SC = post-dispatch losses. Non-overlapping.

3 architecture blockers (from LLD review, blocking milestone M3):

- F1 — BLOCKER:** `instock_corrected_drr` has no data source. `stg_inventory` holds only the latest ATP snapshot, not 14-day hourly ATP history. The DRR formula needs history to exclude OOS hours from the denominator. Fix: emit NULL for this column in V1.
- F2 — BLOCKER (correctness + performance):** `stg_movement_inputs` is read twice — once by J2 for quantities, again by J3 for status labels. This is the largest staging table. Fix: have J2 add the SC label columns to its output so J3 doesn't need the second full scan.
- F3 — HIGH:** `source_clusters` JSON is exploded per-row in J3 via a UDF (User Defined Function — custom code that Spark calls for each row). J1 passes the JSON blob intact and J3 explodes it in the hottest node. Fix: pre-explode in J1 and pass structured columns instead. This is the single biggest compute cost in the pipeline.

Branch divergence:

- `master` branch = real working code (full 9-node Spark pipeline, complete LLD, Azkaban DAG config) but missing Claude harness agents

- `main` branch = generic scaffold (harness agents, settings) but missing actual Spark code and the up-to-date docs
- `meridian_review` = most recent, has the expert architecture review with the 3 blockers

Still open (7 items needing PM confirmation):

1. FSN Portfolio taxonomy options — PRD says Dry/DBEFM, prototype shows Dry/Frozen/Chiller/Ambient
2. Demand MAPE bucket scale — PRD uses 100-125%→>200%, prototype uses 15-30%→>100%
3. MAPE formula — B7.3 uses actual/forecast ratio, C1 uses $|\text{Anticipated}-\text{Actual}| \div \text{Actual}$
4. `actual_sales` valid-status filter — which `order_status` values count as confirmed
5. Store OPD source field — exact column in `orders_fact`
6. S2S flag in `cron_output_fact` — whether it's already emitted
7. Capacity Pivot exact values — SH level or lane level

Key derived metrics (how the main dashboard numbers are actually computed):

- $\text{Instock \%} = \text{COUNT}(\text{FSN} \times \text{DS} \text{ where ATP} > 0) / \text{COUNT}(\text{all active FSN} \times \text{DS})$
 - $\text{Intransit Instock \%} = \% \text{ of FSN} \times \text{DS pairs where ATP} > 0 \text{ OR open STN in transit}$
 - $\text{Inventory Aware Deficit} = \min(\text{Deficit}, \text{Available Inventory at SH at run time})$ — the "actually fulfillable" shortfall
 - $\% \text{ Allocation vs Inventory Aware Deficit} = \frac{\text{allocated_qty}}{\text{inv_aware_deficit}} \times 100$ — the core planning health metric
-

Part 7: What YOU Are Building vs What Exists

Already exists (from old IPC RCA / zero-ops-skynet):

- The RCA computation formulas (3 checks)
- Bulk export via Azkaban + ipc_requests + D42 presigned URLs
- MySQL as a serving store

Already exists (in the meridian repo `master` branch):

- Full 9-node Spark pipeline code (in `instock-spark/`)
- Complete LLD (1,714 lines), NODE_CONTRACTS, RUNBOOK, DATA_QUALITY_RULES
- 7 node JSON manifests in `instock-spark/nodes/`
- Azkaban DAG config

What's new / needs to be built for the 3 dashboards:

- Elasticsearch indexing (J6 dual-sink)
 - The `instockctl-api` K8s microservice (REST API layer)
 - The 4th RCA bucket (STN Drop-Off) — ported from the split design
 - Snapshot history + delta computation (J5)
 - Anomaly gate
 - The two-lane async export router
 - The UI contract (your Confluence page covers this)
-

Part 8: Data Flow Summary (end to end, in plain English)

1. Raw data arrives: CSVs are uploaded to GCS, production databases emit changes via Kafka/binlog
 2. Zero-ETL and Flink CDC convert everything to Iceberg tables on GCS
 3. Every ~2 hours, the Azkaban DAG kicks off
 4. `instock_prep` checks all 16 sources are fresh → if not, pipeline stops
 5. J1 reads all sources, filters to Minutes universe, writes 5 staging tables
 6. J2 reads STN/consignment data → computes quantity waterfall → writes movement fact
 7. J3 reads everything → builds OOS universe → runs 4 root-cause checks → writes 6 output tables
 8. Verification gate runs 20 quality rules → blocks bad data
 9. J4 rolls up to DS / City / subpivot levels
 10. J5 takes a snapshot → computes deltas (vs last run, vs 6AM, vs last week)
 11. Anomaly gate checks for suspicious drops → blocks if threshold exceeded
 12. J6 dual-writes: MySQL gets full detail, ES gets pre-aggregated tiles
 13. API serves dashboard: ES for tiles/lists/search, MySQL for drilldowns
 14. User clicks Export → two-lane router → small=inline, large=Spark → presigned GCS download
-

125 data points across 3 dashboards. 16 sources. 9 pipeline nodes. 4 root-cause buckets. 2 serving stores. 1 API. That's Meridian.